arXiv:2412.10872v2 [cs.CR] 21 Dec 2025

ThreatPilot: Attack-Driven Threat Intelligence Extraction

MING XU, National University of Singapore, Singapore
HONGTAI WANG, National University of Singapore, Singapore
JIAHAO LIU, National University of Singapore, Singapore
XINFENG LI, Nanyang Technological University, Singapore
ZHENGMIN YU, Fudan University, China
WEILI HAN, Fudan University, China
HOON WEI LIM, Cyber Special Ops-R&D, NCS Group, Singapore
JIN SONG DONG, National University of Singapore, Singapore
JIAHENG ZHANG, National University of Singapore, Singapore

Efficient defense against dynamically evolving advanced persistent threats (APT) requires the structured threat intelligence feeds, such as techniques used. However, existing threat-intelligence extraction techniques predominantly focus on individual pieces of intelligence–such as isolated techniques or atomic indicators–resulting in fragmented and incomplete representations of real-world attacks. This granularity inherently limits on both the depth and the contextual richness of the extracted intelligence, making it difficult for downstream security systems to reason about multi-step behaviors or to generate actionable detections. To address this gap, we propose to extract the layered Attack-driven Threat Intelligence (ATIs), a comprehensive representation that captures the full spectrum of adversarial behavior. We propose ThreatPilot, which can accurately identify the ATIs including complete tactics, techniques, multi-step procedures, and their procedure variants, and integrate the threat intelligence to software security application scenarios: the detection rules (i.e., Sigma) and attack command can be generated automatically to a more accuracy level. Experimental results on 1,769 newly crawled reports and 16 manually calibrated reports show ThreatPilot's effectiveness in identifying accurate techniques, outperforming state-of-the-art approaches of AttacKG by 1.34× in F1 score. Further studies upon 64,185 application logs via Honeypot show that our Sigma rule generator significantly outperforms several existing rules-set in detecting the real-world malicious events. Industry partners confirm that our Sigma rule generator can significantly help save time and costs of the rule generation process. In addition, our generated commands achieve an execution rate of 99.3%, compared to 50.3% without the extracted intelligence.

Additional Key Words and Phrases: Cyber Threat Intelligence, Sigma Rule Generation, Log Analysis, Software Security Application

1 Introduction

Advanced Persistent Threat (APT) attacks are serious attacks in the modern world, causing massive losses. APT attack behaviors often leave traces in various organization-internal log monitoring systems, such as system or web application logs. To counter APT attacks, Security Operation Centers (SOCs) typically rely on Security Information and Event Management (SIEM) systems [10, 40, 42] to continuously collect and correlate these security events/logs, carrying out threat steps for real-time incident response. Beyond internal threat logs, external textual sources—such as blog posts (e.g.,

Authors' Contact Information: Ming Xu, National University of Singapore, Singapore, Singapore, mingxucs18@gmail.com; Hongtai Wang, National University of Singapore, Singapore, Singapore, wanghongtai0702@gmail.com; Jiahao Liu, National University of Singapore, Singapore, Singapore, jiahao99@comp.nus.edu.sg; Xinfeng Li, Nanyang Technological University, Singapore, Singapore, lxfmakeit@gmail.com; Zhengmin Yu, Fudan University, Shanghai, China, zmyu23@m.fudan.edu.cn; Weili Han, Fudan University, Shanghai, China, wlhan@fudan.edu.cn; Hoon Wei Lim, Cyber Special Ops-R&D, NCS Group, Singapore, Singapore, hoonwei.lim@ncs.com.sg; Jin Song Dong, National University of Singapore, Singapore, Singapore, dcsdjs@nus.edu.sg; Jiaheng Zhang, National University of Singapore, Singapore, Singapore, jhzhang@nus.edu.sg.

2025. Manuscript submitted to ACM

Cisco Talos [7]), government advisories (e.g., DARPA TC [2]), open-source knowledge bases (e.g., `MITRE ATT&CK` [12]), security forums, and analyst reports—also provide Cyber Threat Intelligence (CTI) that highlights key attack elements such as techniques used.

Raw CTI reports are typically in natural language paragraphs, loosely structured and contain much redundant information, making them difficult to interpret and act upon. To facilitate rapid dissemination of threat understanding and effective cross-team collaboration, security analysts typically abstract the threat intelligence from the unstructured report, e.g., a hierarchical intelligence structure like Tactics, Techniques, and Procedures (TTPs). Well-structured threat intelligence can enable and enhance several core security threat detections and understandings, including:

- **Sigma Rule Configuration:** The real-time demand for web security makes SIEM systems and their applied security rule an integral part of the intrusion detection life-cycle. Sigma provides a standardized and vendor-agnostic format to define detection constraints of malicious events from chaotic logs within most SIEM systems (e.g., Splunk [40], IBM QRadar). An effective Sigma rule configuration relies on accurate threat intelligence, e.g., the `detection` and `tag` fields depend on the Procedures and Tactic-Technique pairs.
- **Attack Behavior Validation:** Generating reproducible attack scripts becomes crucial whenever analysts must recreate real threat behaviors to test defenses, validate rules, or ensure consistent benchmarking. Security experts and red teams often generate reproducible attack commands (e.g., typical spear-phishing PowerShell scripts), to simulate attack paths, understanding the attack dependencies and boundaries. The effective and correct commands is bounded to a series of implementation steps (i.e., attack procedures).

1.1 Existing Methods and Limitations

We investigate existing researches for threat intelligence extraction and present a consolidated summary in Table 1. However, we summarize that prior works like TTPDrill [19] or ThreatRaptor [17] have typically extracted a subset of individual threat intelligence—most commonly techniques—leaving out richer elements such as procedures, and procedure variational behaviors or explanations. This narrow focus limits both the depth and completeness of the resulting intelligence, making it difficult for software security systems to accurately reconstruct attack chains, assess adversary capabilities, or support high-fidelity detection engineering. Nowadays, real-world cyberattacks have recently evolved and often become complex, which can appear benign when viewed in isolation, while multi-step attack chains unfold across multiple related events. Besides, STIX, the TTP extraction approaches (in Table 1) and even recent efforts of CTINexus [11] translating reports into knowledge graph representations, all assume that all threat intelligence are explicitly included in original report, neglecting implicit processes of attack variants. Such process variants, although not directly mentioned, are often key pathways used by advanced attackers to evade detection [28, 31].

In addition, many existing approaches rely heavily on generic NLP pipelines, which often struggle with domain-specific terminology, subtle semantic cues, and cross-sentence dependencies. As a result, technique identification becomes noisy and error-prone, leading to incomplete or inconsistent coverage of real attacks. The inaccurate threat intelligence leads to the poor Rule/Command Adaptability. High-quality Sigma rules rely on accurate and comprehensive threat intelligence, particularly detailed knowledge of techniques and the procedures used to carry them out. Techniques provide the high-level "what" of an adversary's behavior, allowing rule authors to target the correct tactic or behavioral pattern. However, procedures capture the concrete "how"—the specific commands, tool invocations, parameter choices, and execution sequences that adversaries actually use in practice. Without precise procedure-level intelligence, Sigma rules often become overly generic, miss critical variants, or fail to trigger on realistic attack traces. Conversely, when

Table 1. Approaches on threat intelligence extraction. NLP: natural language processing; IR: information retrieval.

	Tactic	Technique	Procedure	Procedure Variant	Software Security Task Adaption	Methods	Year
iACE [25]	×	×	×	×	×	Regex	2016
TTPDrill [19]	✓	✓	×	×	×	NLP and IR	2017
rcATT [20]	✓	✓	×	×	×	Supervised learning	2020
EXTRATOR [37]	✓	×	×	×	×	NLP	2021
ThreatRaptor [17]	×	✓	×	×	×	Unsupervised learning	2021
AttacKG [23]	×	✓	✓	×	×	Knowledge graph	2022
CTINexus [11]	×	×	×	×	×	LLMs	2025
ThreatPilot	✓	✓	✓	✓	✓	LLMs	–

∗ Variant refers to a procedure that is new but plausible inference beyond the original report based on a grounded technical database

techniques and procedures are well understood, rule authors can design more discriminative conditions, reduce false positives, and ensure the rules cover real-world attack paths rather than only abstract tactics. As a result, accurate threat intelligence directly determines the effectiveness, robustness, and operational value of Sigma-based detection.

Together, these limitations reduce the practical usefulness of extracted CTI, hinder automation, and weaken the reliability of security analytics that depend on high-quality threat intelligence.

1.2 Our Solution: ThreatPilot

Drawing from both a systematic review of prior works and our own experience of building and evaluating CTI pipelines, we arrive at building the whole threat intelligence analysis and integration. First, we propose **attack-driven threat intelligence (ATI)** –a new structured format including the complete tactic, technique and procedure (TTP) chains, implicit procedure variants and explanations. The variants, which are although not explicitly mentioned in reports, but are grounded based on pluggable threat database, strengthening the detection coverage beyond observable behaviors [28, 42]. Second, with the ability of large language models (LLMs) to understand unstructured language and perform complex reasoning [13, 22, 26], we deploy LLM-powered pipeline to improve the precision and depth of intelligence extraction, and the adaptability of rules and commands. We integrate these components into ThreatPilot — a system designed to extract ATIs and improve adaptability for software security tasks. ThreatPilot addresses the limitations by: i) To improve the technique extraction accuracy, we combine a semantic *chunker* algorithm, a combined component of *Knowledge Enhancer*, and a *Validator* to solve the problem of missed techniques and those incorrectly identified techniques. ii) ATI inference is guided by a mapping mechanism that integrates an external database with a graph, enabling ThreatPilot to infer plausible behaviors beyond those explicitly described based on the connecting multiple pieces of threat information, linking dispersed evidence such as log entries, tool usage and targeted assets, keeping inferences grounded in external knowledge. iii) ThreatPilot can leverage the proposed attack-driven intelligence to significantly enhance downstream software security tasks such as Sigma rule generation and command construction. It embeds a set of mapping functions that align the extracted ATIs—such as techniques, procedures, parameters, and contextual cues—with the corresponding fields required in Sigma rules and executable attack commands. This grounding allows ThreatPilot to produce detection logic and commands that are not only syntactically correct but also semantically tied to real adversarial behaviors. The initial Sigma rules and attack commands then autonomously interact with syntax-checking modules (e.g., Sigma Lib, Python AST) and external live systems (e.g., Splunk [40]) to validate grammar, refine rule conditions, and adapt configurations to different environments.

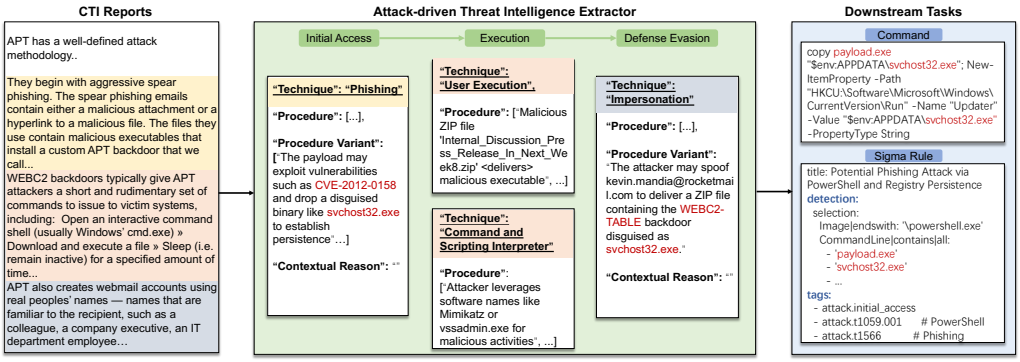


Fig. 1. A demonstrating example of ThreatPilot. The prior works show individual intelligence while ThreatPilot exhibits the whole attack-driven intelligence including three attack stages (e.g., Initial Access), with their techniques (e.g., Phishing), procedures and variants, and contextual explanations. The Sigma rule and command are derived from the "phishing" technique.

By iteratively enforcing correctness and system compatibility, ThreatPilot transforms raw attack intelligence into operational artifacts that are tuned for accuracy, robustness, and real-world deployability.

A Demonstrating Example. We show a demonstrating example in Figure 1 when interpreting the snippet of a CTI report [1] into our-proposed ATIs. For example, while the original CTI report may describe a phishing email leading to malware installation, our system can infer that the malware exploits known vulnerabilities such as `CVE-2012-0158` and drops a disguised binary like `svchost32.exe` to establish persistence.

The extracted ATI provides richer, procedure-level semantics that align naturally with what downstream software security tasks require, enabling more accurate and context-aware rule and command generation. Sigma rules [38] are fundamentally pattern specifications of suspicious or malicious behavior; they rely on precise field–value mappings—such as matching execution entities, process command lines, or behavioral sequences—to function effectively. However, these field values are not arbitrary strings: they must faithfully correspond to concrete attacker actions, tools, and parameters described in real attack procedures. For example, the detection field contains the core functional conditions, which are mapped from the attack execution procedures: the `Image` matches the execution entity, and the `CommandLine` matches the procedures (from the phishing techniques). Other fields might include the title as a brief explanation of what the rule detects, the tags as techniques classifications. In this way, ThreatPilot leverages ATI to bridge a long-standing semantic gap between high-level threat intelligence and the low-level configuration values required by Sigma rules. This mapping ensures that generated rules not only follow syntactic correctness but also faithfully capture the operational patterns of the attack, leading to more robust, precise, and environment-aware detection logic for downstream software security systems.

Results. We deploy ThreatPilot upon GPT-4o, GPT-4o-mini, DeepSeek-V3, and LLaMa3. Experimental results based on 1,769 recently crawled reports [8] and 16 manually calibrated reports [24] show that all deployed ThreatPilot can accurately identify techniques, outperforming the state-of-the-art approach of AttacKG [23] by up to 1.34× in F1 scores. Based on 64,185 application logs collected by a Honeypot in the wild and a Splunk SIEM for rules' executing, we show that the generated Sigma rules by ThreatPilot can catch significantly more malicious events than that by standalone LLMs and the existing rules-set from Sigma [38] and Splunk [41]. We evaluate the generated commands to verify their

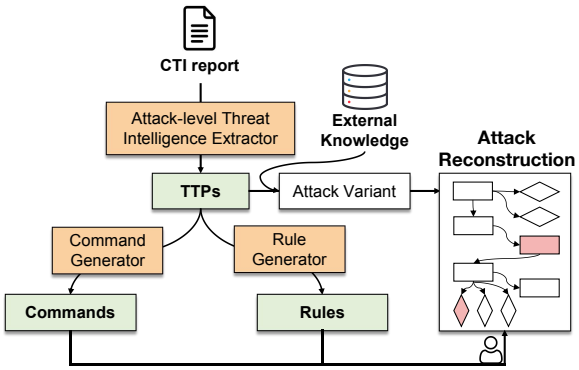


Fig. 2. General overview of ThreatPilot, which includes ATI extractor, command generator, and rule generator

executability: the commands by ThreatPilot achieves up to 99.3% execution rate, which is significantly higher than the 50.3% of commands generated without procedures.

We summarize our contributions as follows.

- **Attack-driven Threat Intelligence.** We propose a novel representation of ATI, which consists of whole TTPs and the procedure variants with explanations. We design ThreatPilot to accurately extract ATIs, which are used to guide the generation of Sigma rules and commands.
- **Extension Evaluation.** We evaluate ThreatPilot upon large-scale CTI reports, demonstrating its higher ATI extraction accuracy and down-stream tasks' adaptability. Tested upon real-world logs, the Sigma rules generated from ThreatPilot can significantly outperform that generated from general LLMs, and the static rules-set in detecting live malicious events.
- **Real-world deployment.** We deploy our generated Sigma rules upon real-world web application logs collected via a honeypot, showcasing their better effectiveness that existing rule-sets in detecting real-world malicious events.

We commit to release all our used datasets and codes for reproducibility of data and results.

2 Design of ThreatPilot

Overview of ThreatPilot. Given an unstructured CTI report as input, ThreatPilot (shown in Figure 2) first extracts structured attack-driven threat intelligence (ATI), including TTPs and attack variants, which are then fed into ThreatPilot to generate Sigma rules and reproducible commands.

2.1 Attack-driven Threat Intelligence Extractor

Given an unstructured report $\mathcal{R}$, and a technique database $\mathbb{T} = \{(t_i, d_i)\}_{i=1}^{N}$ consisting of N techniques t_i and their corresponding standard descriptions d_i. The first goal is to extract a subset of relevant techniques $\mathbb{T}_{\mathcal{R}}$ mentioned in $\mathcal{R}$.

$$\mathbb{T}_{\mathcal{R}} \subseteq \{t_i\}_{i=1}^{N}$$

Given the extracted techniques $\mathbb{T}_{\mathcal{R}}$, the original report $\mathcal{R}$, and an external procedure database $\mathcal{P}$, our goal is to generate a set of procedures and their variants using a generation function.

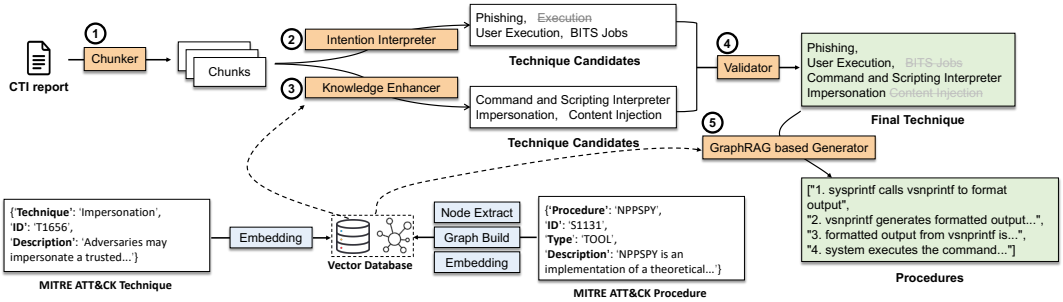


Fig. 3. Overview of attack-driven threat intelligence (ATI) extractor.

$$\hat{P} = \{p_j\}_{j=1}^{M}$$

We follow the top-to-down strategy with a stepwise execution, sequentially outputting each intelligence by using the output of o_i (i.e., tactics-techniques) as contextual input for o_{i+1} (i.e., procedures), to ensure an progressive intelligence extraction. We use the technique database from `MITRE ATT&CK` [1], as it serves as a standardized reference for the security community, providing a globally accessible taxonomy of real-world adversary tactics and techniques.

2.1.1 <Tactic, Technique> Identification. As shown in Figure 3, to enable an accurate identification, we first propose a chunker component ① to decompose $\mathcal{R}$ into multiple smaller chunks, which are expected to contain attack behaviors. The chunks are passed into an ② intention interpreter to extract the technique candidates via the in-context learning abilities of LLMs, followed by a ③ knowledge enhancer to strength more techniques via a external domain-specific database, concluded by ④ validator to judge the technique candidates generated by previous steps as the final recognized techniques.

CTI Chunker. We first propose a CTI chunker that segments a long report into smaller and meaningful chunks, with the goal of ensuring that a chunk retains critical intelligence, eliminating irrelevant information. This is because that LLMs face inherent performance limitations with long inputs due to inefficiencies in the attention mechanism and information loss across long sequences [16, 33, 44, 45].

As outlined in Algorithm 1, our algorithm first identifies that those blocks (e.g., N sentences) containing IoC (Indicators of Compromises) entities, such as malicious IP addresses or suspicious file hashes. We then merge the surrounding (i.e., preceding and following) sentences with the blocks as the final chunks, with the granularity controlled by the parameters of $block_{size}(N)$ and $MergedSentence_{size}$. The core principle is that IoCs represent fundamental indicators of malicious activity and encapsulate critical threat intelligence; blocks lacking IoCs are likely irrelevant to actual threats. To extract IoCs, we design a few-shot prompt, with specified tasks stating that the task is to extract IoCs such as IP addresses, domains, URLs, file hashes, and email addresses. We don't consider traditional methods such as regular expressions, pattern matching, dictionary-based lookups for extracting IoCs, as they always lack semantic understanding, leading to false positives, e.g., extracting file hashes simply based on their format, without considering surrounding context.

[1] https://attack.mitre.org/

Algorithm 1 <Tactic, Technique> Identification

Input: CTI report $\mathcal{R}$; vector database $\mathcal{D}$
Output: Identified <tactic, technique> pairs $\mathbb{T}_R$

// Step 1: CTI Chunking
$\mathcal{C} \leftarrow \{\}$ // Chunked report set
Split $\mathcal{R}$ into m blocks $b_1, \ldots, b_m$, each with N sentences
for each $b_i \in \{b_1, ..., b_m\}$ **do**
  if LLM_ContainsIoC(b_i) **then**
    $r_i \leftarrow$ MergeSentencesAround(b_i, $MergedSentence_{size}$) // Expand block to include surrounding context
    $\mathcal{C} \leftarrow \mathcal{C} \cup \{r_i\}$ // Add chunk r_i to the valid chunk set
  end if
end for
// Step 2: Intention Interpreter
for each $r_i \in \mathcal{C}$ **do**
  $q_i \leftarrow$ CoT_Prompt(r_i) // Generate reasoning output using CoT
  $\hat{\mathbb{T}}^{\text{CoT}}_{r_i} \leftarrow$ Parse_CoT(q_i) // Parse predicted tactic-technique pairs
end for
// Step 3: Knowledge Enhancer based on Vector Database Retrieval
for each chunk $r_i \in \mathcal{C}$ **do**
  $E_{r_i} \leftarrow \mathcal{V}(r_i)$ // Compute embedding vector of chunk r_i
  $\mathcal{S}_i \leftarrow \{\mathcal{S}(E_{r_i}, \mathcal{V}(d_j)) \mid d_j \in \mathcal{D}\}$ // Compute similarity scores
  $\hat{\mathbb{T}}^{\text{Vec}}_{r_i} \leftarrow$ TopK($\mathcal{S}_i, k$) // Select top-k most similar techniques
end for
// Step 4: Validation and Consolidation
$\mathbb{T}_R \leftarrow \emptyset$ // Initialize final result set
for each $t_i \in \bigcup_i \left(\hat{\mathbb{T}}^{\text{CoT}}_{r_i} \cup \hat{\mathbb{T}}^{\text{Vec}}_{r_i}\right)$ **do**
  $d_i \leftarrow$ GetDescription(t_i) // Retrieve standard MITRE description
  $verdict_i \leftarrow F(d_i, \mathcal{R})$ // Use independent LLM-as-a-judge to assess semantic alignment
  if $verdict_i = true$ **then**
    $\mathbb{T}_R \leftarrow \mathbb{T}_R \cup \{t_i\}$ // Retain the technique
  end if
end for
return $\mathbb{T}_R$

Intention Interpreter. With the segmented chunk-list $\mathcal{C} = \{r_1, r_2, ..., r_i\}$, we employ a two-stage intention interpreting framework by first performing *Tactic Planning* to identify high-level attacker objectives, followed by *Technique Generation* to derive specific techniques. The Chain-of-Thought (CoT) reasoning process can be formally represented as

$$\mathcal{T}_{r_i} \leftarrow \text{Tactic_Plan_Prompt}(c_i)$$

where Tactic_Plan_Prompt is a guided CoT prompt designed to elicit reasoning about attack intent, constrained to the 14 official tactics in `MITRE ATT&CK`. The LLM is instructed to infer a plausible tactic plan based on the semantics of the input chunk. For each identified tactic $\tau \in \mathcal{T}_{r_i}$, we further generate candidate techniques that are conditioned on both the original chunk and the given tactic:

$$\hat{\mathbb{T}}^{\tau}_{r_i} \leftarrow \text{Guided_Generation}(c_i, \tau)$$

where Guided_Generation incorporates a hybrid processing including LLM generation and rule-based filtering. We leverage a curated set of tactic-specific rules that link tactics to indicative keywords or behavioral patterns. If a sentence in the chunk matches the associated rules, we trigger a tactic-aware prompt to generate relevant techniques.

To mitigate hallucination and ensure factual alignment with `MITRE ATT&CK`, we perform a post-processing step to validate the generated techniques. Each technique is checked against the official technique list. If a generated technique is not found, we compute its embedding and retrieve the top-3 most similar techniques within the same tactic. A replacement is made if a semantically aligned technique is identified; otherwise, the hallucinated output is discarded.

Knowledge Enhancer. To enhance the extraction of missed techniques, we incorporate a retrieval-augmented generation (RAG) strategy [21] that dynamically compare the standard technique descriptions with the input chunks. The RAG knowledge base is bootstrapped from MITRE's official documentation [12], which provides detailed illustrations for the technique's meaning. From this corpus, we build the the vector database storing key-value pairs of <tactic-technique: description>, where descriptions are projected into an embedding space for comparison of similarity with input chunks. Given that lengthy standard descriptions (e.g., spanning three or four paragraphs) may introduce information loss, we calculate the average word count of the first paragraph and use this value (i.e., 128 tokens) to segment the standard descriptions into several d_j. Finally, the chunks are embedded and the dot product is calculated with all the vectors in vector database.

In this way, more techniques can be identified by

$$\hat{\mathbb{T}}^{\text{Vec}}_{r_i} = \arg\max_{i \in \mathbb{T}} \text{avg}\left(Top_k[\mathcal{S}(\mathcal{V}(r_i), \mathcal{V}(d_j)]\right)$$

where $\mathcal{V}(\cdot)$ denotes the vector embedding model (`text-embedding-ada-002` [4] in our task), $S(\cdot,\cdot)$ denotes the function to compute the similarity score, $\mathbb{T}$ is the set of all techniques, i.e., 235 techniques in `MITRE ATT&CK`. The $Top_k(\cdot)$ is the function that picks the k similar techniques, which are returned as the identified technique candidates.

Table 2. Prompt template for technique classifications. Our prompt is structured into four key components: Background, Task, Guidelines, and optional examples as a progress approach for analyzing, refining, and integrating information.

Background: You are a helpful assistant with the security domain knowledge.
IoC Recognition for <CTI Chunker>: Task: Your task is to extract the indicators of compromise (IoCs) from the following CTI report. Examples: IP Address, Domains, URLs, Email Addresses; File Hash, URL, Process Names, Registry Keys. **Final prompt:** Background + Task + Guidelines + Examples + $Chunks | Report
<Intention Interpreter>: Task: Identify the high-level ATT&CK tactics that the threat actor is likely pursuing in the order of execution. Use only tactics from the official `MITRE ATT&CK` list. Below are the tactics' names and descriptions: {TACTIC_DESCRIPTION} Based on the tactic, list the `MITRE ATT&CK` techniques this behavior likely represents. Below is the complete mapping of each tactic to its associated techniques: {TACTIC_TECHNIQUES_MAPPING} **Final prompt:** Background + Task + Guidelines + $Chunks | Report
<Knowledge Enhancer>: Task: According to the report and the vector database, find the techniques most relevant to the description **Final prompt:** Background + Task + $Chunks | Report
<Validator>: Task: The user will provide another possible technique and description. You need to verify whether the technique exists in the report. **Final prompt:** Background + Task + Guidelines + $Technique Candidates: Standard Descriptions + $Report

Validator. The technique candidates generated in previous two steps might be irrelevant to the report, leading to false positives. To mitigate this, we introduce a validator to ensure that most responsible and correct techniques are finally settled down. Specifically, validator employs a custom LLM, isolated from prior context, to judge the technique candidates, removing the overly general techniques. The validator follows the grounded reasoning by re-thinking and comparing two descriptions between the entire report $\mathcal{R}$ and the descriptions of every technique candidates d_i, as formalized below.

$$T_{final} = \{t_i \in T_{candidates} | F(d_i, R) \implies true\}$$

where $F(\cdot,\cdot)$ is a function to distinguish whether the semantics and intention align between two texts. The function $F(\cdot,\cdot)$ is achieved by a validator prompt with detailed guidelines. Note that the LLM used in *LLM-as-a-judge* operates independently, with no shared memory, and is assigned different system messages to ensure they remain focused on their specific subtasks.

2.1.2 Procedure Generation. To enable the capabilities of modeling relationships between entities for procedures and their variants, we employ GraphRAG [15], which specializes in the joining operations via the built-in graph structure. Instead of the RAG, we specifically design a multi-hop GraphRAG-based reasoning modular to estimate the relationships among entities. To achieve this, ThreatPilot first performs a report narrowness to find a meaningful part (i.e., procedure paragraph) given the technique guidance. The procedure paragraph is linked through the snippet of external vector database and the original report via reasoning; second, ThreatPilot decouples the procedure paragraph into procedures. Formally, given a CTI report $\mathcal{R}$, the procedure generation is defined as:

$$\hat{P} = \text{Gen}\left(Integrate(\mathcal{R}_{\text{ent}}, \mathcal{R}_{\text{rel}}, \mathcal{R}_{\text{comm}}, \mathcal{R}_{\text{sum}})\right)$$

where the fused context combines four types of retrieved information: entity-level mentions ($\mathcal{R}_{\text{ent}}$), relationship paths ($\mathcal{R}_{\text{rel}}$), community clusters ($\mathcal{R}_{\text{comm}}$), and high-level summaries ($\mathcal{R}_{\text{sum}}$). These are jointly used by the generation module $\text{Gen}(\cdot)$ to produce the final procedures $\hat{P}$.

ThreatPilot dismantles the procedure paragraph (shown below) into several procedures by reasoning the relevant entities and relationships, where the procedure paragraph is referenced from the external reports with their array numberings (as exampled below). When the model encounters a partially described procedure in the CTI report, it leverages the graph-based retrieval to infer related steps, tools, or IoCs by identifying similar entities and neighborhood structures. ThreatPilot matches extracted IoCs and entities in the report with nodes in the knowledge graph and retrieve their adjacent nodes (e.g., common follow-up actions or co-occurred artifacts). This guided retrieval allows the model to generate faithful variants that may not be explicitly mentioned in the text but are supported by the knowledge database, rather than hallucination from the LLMs. ThreatPilot uses a classification function $f_{\text{variant}}(\hat{P}, \mathcal{R})$ to identify whether a generated procedure $\hat{P}$ constitutes a novel *attack variant*, providing multiple levels of similarity matching to assess whether $\hat{p}$ can be traced back to $\mathcal{R}$. ThreatPilot segments $\mathcal{R}$ into a sentence set

$$\mathcal{S} = \{s_1, s_2, ..., s_n\}$$

then, encode each sentence into an embedding set

$$\mathcal{E}_{\mathcal{R}} = \{e_1, e_2, ..., e_n\}$$

ThreatPilot employs normalized string matching, semantic similarity based on sentence embeddings, and keyword overlap. A procedure is classified as an attack variant if it cannot be sufficiently matched to any sentence in the report. We consider a procedure as matched if it either has high lexical overlap, exceeds a semantic similarity threshold (e.g., 0.84), or appears nearly verbatim in the text.

Parameters. The vector database consists of 677 procedures in `MITRE ATT&CK`, in the format of ⟨*technique: procedures*⟩. We input the whole report into GraphRAG, because it has the internal chunking mechanisms. The generated knowledge graph comprises 6,002 entities and 10,900 relationships within 36 categories. A category refers to a set of entities and paragraphs with semantically consistent and contextual relevance.

An example of the procedure generation process

Technique: Command and Scripting Interpreter
Procedure Paragraph: The primary technique utilized in the attack is the **Command and Scripting Interpreter**, specifically through the use of **Windows' cmd.exe**. The use of cmd.exe is a common tactic in cyber attacks, enabling attackers to leverage built-in system functionalities for malicious purposes [Data: Reports (590, 728, 501, 463, 520)].
Procedure: cmd.exe opens interactive command shell

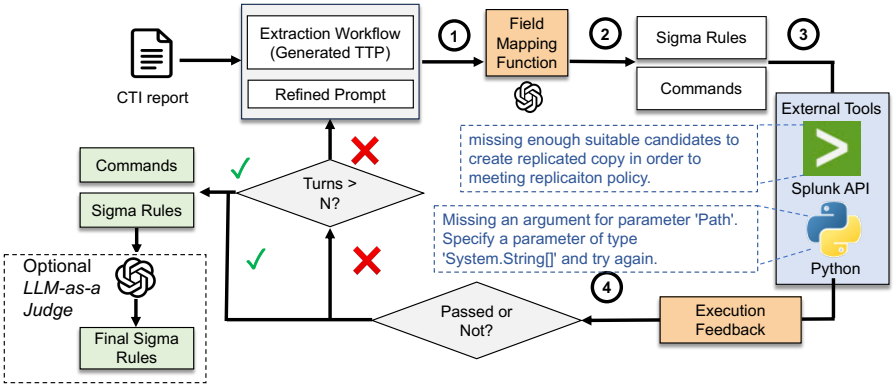


Fig. 4. Overview of Sigma Rule/Command Generator.

2.2 Generator for Sigma Rules and Commands

ThreatPilot integrates the ATI as the initial CoT reasoning prompts as the primary semantic driver, and then combine a system integration module linking external tools (e.g., Splunk API or Python AST), enabling iterative feedback and refinement (shown in Figure 4) to generate Sigma rules and commands.

2.2.1 Sigma Rule Generator. ThreatPilot employs the functional mapping that the specific parameter value of every fields (keywords) in a Sigma rule is purposely guided and grounded in corresponding ATIs. The core detection field is fully grounded in the *procedure* or *procedure variant* description, with field-value pairs reflecting key entities such as command lines, image names, registry paths, and network indicators. For example, if the procedure specifies `powershell.exe -enc ...` to establish persistence, then the rule's detection should reflect conditions on: `Image=powershell.exe`, `CommandLine contains '-enc'`. We employ a semantic prompt with the template in Table 3 to achieve the mapping functions above, where the rules can be tailored in Guidelines when the down-stream tasks unavoidably should include domain-specific or application-specific symbols.

To maintain the syntax and semantic consistencies, ThreatPilot dynamically invokes modules via Splunk API to address identified issues when converting our generated Sigma rules into Splunk, chosen for its popularity [40, 41]. Formally, let $R = \{r_1, r_2, \ldots, r_n\}$ represent a set of Sigma rule candidates. $f_{\text{yaml}}(r)$ be the function that checks if the rule r is in valid YAML format. $f_{\text{sigma}}(r)$ be the function that checks if rule r follows Sigma syntax. $g(r)$ be the function that converts rule r to a Splunk rule. $f_{\text{splunk}}(g(r))$ be the function that checks the syntax of the corresponding Splunk rule. $M()$ represents message LLMs used to regenerate a rule when validation fails. ThreatPilot applies a series of the following reflection functions to iteratively refine the generated Sigma rules, ensuring grammatical correctness and enhancing semantic accuracy, with the goal of reducing false positives and improving true positive rates.

Table 3. Prompts for Sigma rule generator.

<**Candidate Sigma Rule Generation**>:
Task: For each attack description, write several Sigma rules to detect the relevant attacks. The input is: $Report. ATIs: {$ATIs}
Guidelines : Use the following mappings to transform each procedure and its context into a valid Sigma rule: 1. Procedure command / IoC → detection: Populate the detection field with key indicators such as file names, process paths, or network artifacts. 2. Tactic / Technique → tags: Map to tags to describe the detection scope (e.g., attack.execution, attack.t1059). 3. Procedure severity / tactic impact → level Assign rule severity (e.g., low, high) based on the threat level implied by the tactic. 4. CTI source → references: Include links to the original CTI report or known vulnerabilities (e.g., CVEs) for traceability.
<**LLM-as-a-judge**>:
Task: Your task is to filter out irrelevant rules from the generated rules. You will be given content from a CTI report and a rule. You need to determine whether the rule is relevant to the CTI report. The cyber report is: $Report. The rule you need to judge: {$Sigma_rule}
Guidelines : 1. Focus on alignment between the rule's detection field and the report's described actions. Ignore generic or unrelated rules. 2. You should be very tough on the rules. If the rule is not relevant to the CTI report, you should filter it out.

$$
\begin{aligned}
\text{YAML Format: } & f_{\text{yaml}}(r) = \text{False} \implies r' = M() \\
\text{Syntax Check: } & f_{\text{yaml}}(r) = \text{True} \wedge f_{\text{sigma}}(r) = \text{False} \\
& \implies r' = M() \\
\text{Splunk Rule Syntax: } & f_{\text{yaml}}(r) = \text{True} \wedge f_{\text{sigma}}(r) \\
& = \text{True} \wedge f_{\text{splunk}}(g(r)) = \text{False} \\
& \implies r' = M() \\
\text{Final Condition: } & f_{\text{yaml}}(r') \wedge f_{\text{sigma}}(r') \wedge f_{\text{splunk}}(g(r'))
\end{aligned}
$$

The feedbacks contain syntax errors, missing field references, invalid logical conditions, or incomplete value mappings encountered during the translation of Sigma rules into Splunk-compatible queries (e.g., "missing enough suitable candidates to create replicated copy in order to meeting replication policy").

To ensure the precise rules, a validator is additionally employed to to filter out overly generic rules leading much excessive false positives. We empirically observed that rules like `Image IN (*CalculatorApp.exe, *hollow_process_name.exe)`, which detect whether the current process matches *CalculatorApp.exe* or *hollow_process_name.exe*, can be syntactically correct but overly generic, because such rules rely solely on the image field of the process name without considering other critical characteristics such as the file path, parent process, or command-line arguments. ThreatPilot guides such semantic-misalignment by evaluating the relevance of the Sigma rules with respect to the entire CTI reports R.

Environment. We use the syntax validation using Splunklib's dry-run mode [39], allowing to check for grammar errors without executing the query. We use the Sigma-CLI libraries [38] to convert Sigma rule to Splunk rules. We set an attempt limit of reflection to 30 as the maximum number, discarding rules that exceed this threshold, avoiding an endless loop.

2.2.2 Command Generator. The procedures and their variants are inherent nature of explicitly detailing the actions to be executed, aligning with the commands. ThreatPilot maps the `type` field of a procedure to the corresponding execution environment, and extract the `command` field as the core of the generated executable instruction. When procedure contains placeholders (e.g., `payload` or `target`), ThreatPilot substitutes them using predefined templates or contextual parameters derived from the original CTI report. For refinementThreatPilot goes through an external reflection module, interacting with the Python function feedback. The pipeline can be effectively simplified as:

Table 4. Prompts for command generator.

Task: Generate a valid, executable Windows command based on a given procedure description. The input is: $Report. ATIs: {$ATIs}
Guidelines : Use the following mappings to transform the procedure into a valid, executable Windows command: 1. Procedure type → Execution Environment: Selects the appropriate shell (e.g., PowerShell, cmd). 2. Tactic / Technique → Behavior Check: Ensure the command behavior aligns with the described attack goal. 3. CTI report context → Additional Parameters: Use extra context (e.g., file name, domain) to complete or customize the command.

$$\mathcal{R} \xrightarrow{f_{\text{proc}}} P \xrightarrow{f_{\text{cmd}}} C_\text{candidates} \xrightarrow{f_{\text{reflection}}} C$$

where P is intermediate procedure, and C is final command. We do not apply *LLM-as-a-judge* to command generation, as commands are less prone to semantic ambiguity.

Environment. We choose the platform compatible with the Windows environment, including PowerShell, CMD, Bash, Python, and other supported interpreters. We show CoT prompting in the command generation step in Table 4.

3 Implementation and Evaluation Setup

3.1 Attack-driven Intelligence Extractor

Implementation Details. We implement a prototype of ThreatPilot in Python and open-source it for the community and integrate them into the client-side tool [2] using the Streamlit library. We build ThreatPilot upon GPT-4o, GPT-4o-mini, DeepSeek-V3 (671B) [6] and LLaMa-3 (405B) [27] to show the ThreatPilot's adaptability to various LLM backbones. To control generation behavior, we set all LLMs to a temperature of 0.2 for a balance between determinism and flexibility, top-p of 0.1 for controlled diversity, frequency and presence penalties at 0, and a maximum response length of 512 tokens to prevent excessively long outputs.

In algorithm 1's *chunker*, we set $block_{size}(N)$ as 3 sentences and $MergedSentence_{size}$ as 1 sentence, striking a balance between processing speed and meaningful segmentation. For balancing the false positives and false negatives, we set the top k retrieval results from the vector database as 3.

AttacKG Implementation. We reproduced AttacKG with its released code and model checkpoints, leveraging pre-trained Named Entity Recognition models for entity extraction. Specifically, we used spaCy's `en_core_web_sm-3.1.0` model, fine-tuned on sampled `MITRE ATT&CK` examples, with an optimizer of rule-based entity recognizer. For attack technique identification, AttacKG utilized graph alignment with a fuzzy matching algorithm, where the node alignment threshold was set to 0.65, and the graph alignment threshold was set to 0.85.

CTI Report Collection and Labeling. First, we use the open-sourced experimental reports from AttacKG [3]: 16 reports, including 5 attack reports from DRAPA TC and 11 reports regarding widely-occurred attack campaigns. Due to the large-scale updates on `MITRE ATT&CK` (e.g., 81.5% of techniques' descriptions are modified since 2022), many techniques now differ significantly from previous versions [13] (e.g., The technique of `Command and Scripting Interpreter` added scripting languages of PowerShell, Bash, Python than before), causing the original labeled ground truth to become outdated and lose effectiveness over time. To mitigate such influences, we calibrate the open-sourced 16 reports to align with the updated `MITRE ATT&CK`. We hired two staffs who have at least two years of experience in SOCs for technique labeling. Disagreements were resolved by expert review. This inter-annotator agreement was high (**Cahen's Kappa=0.89**).

[2] https://threatpilot-web.streamlit.app/
[3] https://github.com/li-zhenyuan/Knowledge-enhanced-Attack-Graph/tree/main/Results. The gray and green parts together serve as the ground truth.

We additionally collected 1,769 CTI reports from Cisco as of October 2024 [8], categorizing them into medium (1,398) and severe (371) based on CVSS scores. Medium reports fall within the [4.0–8.9] range, while severe reports range from [9.0–10.0], reflecting the varying threat levels encountered in real-world scenarios. Most reports do not provide unified and formatted threat intelligence to validate our extracted results, which is also our key motivation behind this work. We randomly select several techniques extracted from reports for manual investigation to evaluate their accuracy and reliability.

Metrics for Technique Identification. We calculate the false negatives, false positives, precision, recall, F1 scores. We apply the **exact match criteria** for calculating the metrics:

- **False Negatives (FNs) and False Positives (FPs):** With the ground truth of labeled techniques, false negatives refer to the number of correct techniques that are not identified. False positives refer to the number of identified techniques that are not correct. True Positives (TPs) refer to the number of recognized techniques that are correct.
- **Precision:** the percentage of correct techniques out of all the identified techniques, represented as $\frac{TP}{TP+FP}$.
- **Recall:** the percentage of all correctly identified techniques out of all the correct techniques, represented as $\frac{TP}{TP+FN}$.
- **F1:** the harmonic mean of precision and recall, represented as $F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision}+\text{Recall}}$.

Metrics for Procedure Generation. We evaluate the semantic quality for generated procedures from five perspectives below.

- **Completeness**: whether these procedures follow the logical structure of <entities relationships actions>.
- **Correctness**: whether the procedures are technical errors or security misleading.
- **Relevance**: whether the procedures reflect a real-world or simulated attack scenario.
- **Reproducibility**: can the procedures be reproduced precisely?
- **Variativeness:** whether the generated procedures represent plausible procedure variants?

3.2 Down-stream Tasks

3.2.1 Sigma Rule Generator. To simulate real-world incident response behaviors, we assume that an expert summarizes CTI reports based on attacks observed over a specific period, and these reports are then analyzed for further incident response activities in the subsequent months. To this end, we deployed a honeypot to safely attract and collect real-world application event logs, as rule-based detection is particularly effective for identifying individual/isolated malicious events that are more frequently captured in application logs (e.g., SQL injection, unauthorized access, file uploads). We collect logs over a three-month period from July 22 to October 22, 2024, comprising 64,185 security events. Our experienced analysts summarized CTI reports based on logs collected during the first month, resulting in 32 reports, to test the logs from the following two months.

We compare the following Sigma rules-set in our experiments.

- **ThreatPilot**: Our approach uses ATI-guidance and environment feedback.
- **LLM-generated rules**: Standalone LLMs without extracted ATIs or environmental feedback.
- **Open-source rules:** Rules from public repositories such as Splunk [41] and Sigma [38], filtered for web application attacks and comparable rule count.

Implementation Details. We evaluate detection performance using Splunk version 9.3.1 with a trial license. Logs from August 20 to October 22 were ingested and analyzed by each rule-set. We utilize a Cowrie honeypot [14] deployed on an Amazon Web Services (AWS) server under the three-month free plan. The Cowrie honeypot records the web

attacks [30, 32, 36, 43] in the wild including brute-force attacks (e.g., brute force login via SSH or Telnet to try different combinations.), command injection (e.g., the vulnerable commands), file operation (e.g., modify files to create or alter users, maintaining persistent access to the system) or ransomware (e.g., upload and execute ransomware or encrypt files on the system) attacks, etc. Note that the logs collected by Honeypot are involved with system normal behaviors, as exemplified below, explaining the false positives. Two staffs manually labeled normal security events with Cahen's Kappa of 0.84.

- *2024-07-22,18:19:55,/,"Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/81.0.4044.129 Safari/537.36", 185.254.196.173.*

Metrics. We ingest the test logs into Splunk, execute rules-set as search queries, and verify their accuracy by comparing retrieved logs against the ground truth. Execution is considered successful if the retrieved logs match the expected attack-related malicious logs. We test all the rules-set with the same condition, specifically, we regard the malicious events as those originating from the same labeled malicious IP sources in ground truths.

3.2.2 Command Generator. A command is generated with a well-structured procedure. As a baseline, commands are generated from the same CTI reports without procedural structuring and the limit of command numbers.
Metrics. We use the `subprocess` module in Python, which runs system commands to test the executability of the command. To ensure a fair comparison, we regard a command as executable if it contains no syntax errors (e.g., "syntax error near unexpected token"). Other failures caused by system-specific or environment-dependent factors, such as missing directories, are excluded from our evaluation, eliminating environmental biases. To achieve this, our function calculates the percentage of exit codes, where a *returncode* of 0 indicates none syntax errors. Our evaluation focuses on syntactic validity, as achieving semantic correctness would necessitate subjective methods—such as user studies—which involve high-level behavioral interpretation or integration with threat modeling frameworks. Syntactic validity aligns closely with real-world adversarial behavior, since malicious commands must be syntactically correct in order to execute successfully and achieve their intended effects.

4 Evaluation

In this section, we are keen to investigate the following research questions.

- **RQ1 Accuracy and Depth on ATI Extraction:** How accurately does ThreatPilot identify techniques, and how reliable and useful are the generated procedures/variants from five perspectives?
- **RQ2 Ablation Study:** How does each component (e.g., *knowledge enhancer* and *validator*) enhance the overall performance?
- **RQ3 Effectiveness on Down-Stream Tasks:** How does the detection performance of our Sigma rule generators? What is the execution rate of our generated commands?
- **RQ4 Efficiency:** What is the time latency and economic costs of ThreatPilot?

4.1 RQ1: Accuracy and Depth on ATI extraction

Results on Open-sourced CTI Reports. Table 5 shows the accuracy of ATI extraction on open-sourced CTI reports. ThreatPilot are more accurate in identifying techniques than AttacKG [23] in precision, recall and F1, and upon closed-sourced or open-sourced models. To further evaluate ThreatPilot 's capability in technique extraction, we manually examine the false positives and false negatives that were missed by AttacKG but successfully identified by ThreatPilot, as illustrated in Figure 5. In the `TC_Nginx` report, AttacKG fails to identify the technique of `Exploitation`

Table 5. Accuracy of technique identification. GPT-4o-mini (mini); GPT-4o (4o); DeepSeek-V3 (DS); LLaMa-3 (LMa).

CTI Reports	False Negatives (↓)					False Positives (↓)				
	AttacKG	ThreatPilot				AttacKG	ThreatPilot			
		mini	4o	DS	LMa		mini	4o	DS	LMa
TC_Firefox DNS	2	2	3	4	6	14	2	4	3	4
TC_Firefox Drakon	3	1	2	3	1	4	2	4	1	3
TC_Firefox BITX	5	2	3	4	6	7	3	2	3	3
TC_SSH	5	2	2	3	4	3	1	3	3	3
TC_Nginx	5	4	5	5	4	10	1	2	4	4
Frankenstein	9	3	3	4	6	5	2	4	4	3
OceanLotus	5	3	1	4	2	2	0	1	1	2
Cobalt	10	3	2	4	5	3	2	2	3	2
DeputyDog	10	2	4	2	5	6	1	9	5	5
HawkEye	19	2	4	9	8	8	3	1	6	6
DustySky	5	2	1	3	3	7	3	5	4	6
TrickLoad Spyware	7	2	1	4	2	3	1	3	6	6
Emotet	5	3	1	4	2	4	2	5	4	5
Uroburos	7	3	4	3	5	5	3	5	2	2
APT41	9	4	3	7	4	11	3	6	6	7
Espionage	10	3	4	5	3	3	1	2	4	13
Precision ↑	0.313	0.818	0.696	0.653	0.654	–	–	–	–	–
Recall ↑	0.366	0.767	0.756	0.614	0.625	–	–	–	–	–
F1 score ↑	0.338	0.792	0.724	0.629	0.631	–	–	–	–	–

of Remote Services due to its inability to parse the nounced semantics of remote code execution on the web server's TCP port 80, which aligns with the reports' description. Regarding false positives, The TC_Nginx is unrelated to any user-driven actions, because AttacKG cannot distinguish the fuzzy semantics regarding execution classification.

We attribute this to the fact that ThreatPilot is better at understanding nuanced attack intentions and comprehending complex natural language expressions including ambiguous descriptions and context-dependent information. In contrast, AttacKG's semantic understanding ability is constrained by its training data and reliance on specific keywords and fixed patterns, making it less effective at parsing nuanced semantics.

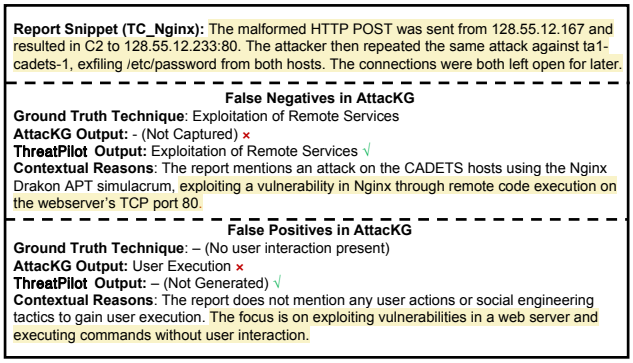


Fig. 5. Examples misflagged by AttacKG but correctly classified by ThreatPilot.

Table 6. Top five techniques identified from recent-crawled CTI reports. M ⇒ Medium report, S ⇒ Severe report.

Rank	Technique	Count
1	`T-1203: Exploitation for Client Execution` (M, S)	940 (M), 318 (S)
2	`T-1068: Exploitation for Privilege Escalation` (M, S)	561 (M), 200 (S)
3	`T-1059: Command and Scripting Interpreter` (M, S)	474 (M), 186 (S)
4	`T-1190: Exploit Public-Facing Application` (M, S)	371 (M), 164 (S)
5	`T-1055: Process Injection` (M) `T-1548: Abuse Elevation Control Mechanism` (S)	299 (M), 78 (S)

Results on Recently-collected CTI Reports. Table 6 lists the five most common techniques and their frequency based on the recent CTI reports. We observe that the distribution of employed techniques in the CTI reports aligns with existing TTP trends [3, 5, 34]. For example, the technique `Command and Scripting Interpreter`, which highlights the use of malicious scripts, is widely adopted across a variety of attacks. Moreover, the top four techniques show significant overlap between medium and severe reports, reinforcing the accuracy of ThreatPilot, as it reflects the realistic scenario where attackers, especially those from similar threat groups, often reuse a common set of techniques. In general, the most frequently occurring techniques tend to remain consistent across different CTI reports, with only minor variations.

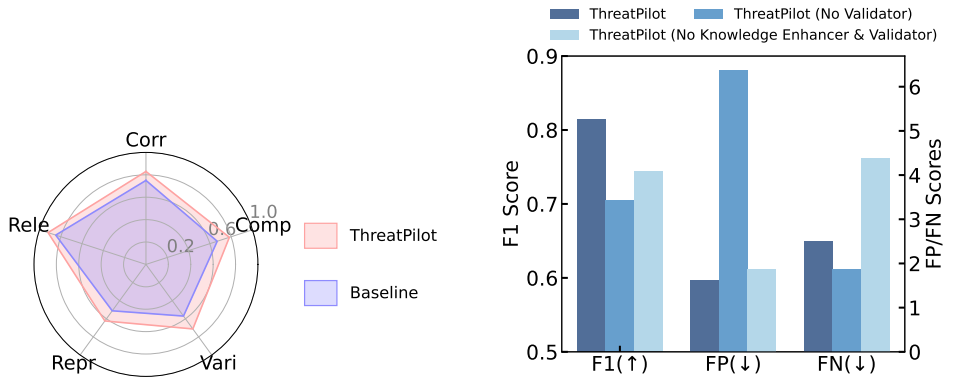
LLM's Comparison. The closed-source foundation models often outperform open-source ones, primarily due to they are pre-trained with larger parameters. Among them, GPT-4o-mini performs best. Furthermore, both GPT-4o and GPT-4o-mini show similar level of false negatives, but GPT-4o generates significantly more false positives. **We recommend using GPT-4o-mini in ThreatPilot.** We speculate that this is due to GPT-4o's creativity to produce more divergent responses, leading to hallucinations and incorrect answers. Given that GPT-4o is around 20x more expensive than GPT-4o-mini while offering minimal additional gains.

Results on Procedure Quality. We qualitatively evaluate the procedures generated by ThreatPilot and that by standalone LLMs (i.e., GPT-4o-mini) without the GraphRAG-based reasoning module. We randomly sample 10% recent severe reports to generate procedures, yielding 1,132 and 314 procedures by ThreatPilot and baseline respectively. We revise our prompts in LLM-evaluator to achieve alignment with human evaluators (Cohen's Kappa = 0.85).

Figure 6(a) presents the average scores per procedure across five evaluation dimensions, demonstrating that ThreatPilot consistently outperforms the baseline in all aspects. Note that ThreatPilot infers the procedure variants by the grounded database embedded in a Graph-based database, making it less prone from the LLM's hallucination. Furthermore, our security experts manually checked that 83.5% (945/1132) are extremely useful based on their daily experience. Take a specific example, the procedures without the GraphRAG-based reasoning components are too general without the entities or relationships, like `Attacker executes commands using the Command and Scripting Interpreter`. With ThreatPilot, the procedures can become more targeted and meaningful, such as `Attacker manipulates system behavior to gain unauthorized access to files outside of the /ltrx_user/ directory`.

4.2 RQ2: Ablation Study

We conducted an ablation study to evaluate the individual contributions of two critical components of *knowledge enhancer* and *validator*. We construct several attack variants: one without both *knowledge enhancer* and *validator*, another without *validator*, and the full version of ThreatPilot that incorporates both components. Our ablation study is based on GPT-4o-mini. We present the results in Figure 6(b), where the full version of ThreatPilot achieves the



(a) Quality ratings between the procedures generated by ThreatPilot and the standalone GPT-4o-mini. Corr → Correctness, Comp → Completeness, Rele → Relevance, Repr → Reproducibility, Vari → Variativeness.

(b) Ablation study: with two components, F1 scores can be significantly enhanced.

Fig. 6. Comparison of procedure quality and ablation study results.

highest F1 score, demonstrating that the combination of the two components leads to improved technique identification performance. The use of the *knowledge enhancer* alone results in lower F1 scores compared to configurations without both *knowledge enhancer* and *validator*, primarily due to the high false positives. Nevertheless, the *knowledge enhancer* plays a crucial role in significantly reducing false negatives. To address the false positive issue introduced by the enhancer, the *validator* module is essential. Together, these two components work synergistically: the *knowledge enhancer* mitigates false negatives, while the *validator* filters out false positives.

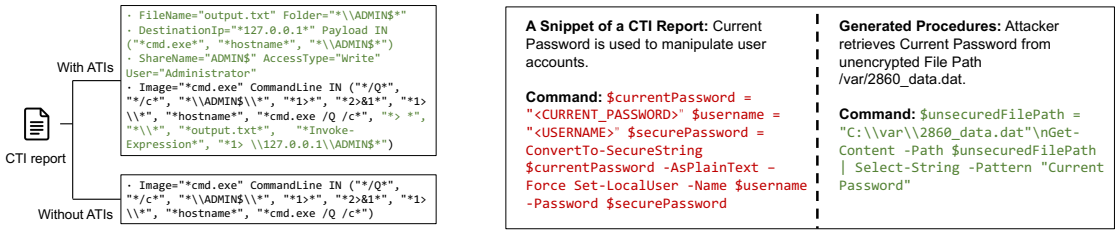
4.3 RQ3: Effectiveness on Down-stream Tasks

4.3.1 Sigma Rule Generator. We systematically evaluate and compare the detection abilities in the wild between the rules-set from ThreatPilot, standalone LLMs and open-source folders. Table 7 presents the results, demonstrating that ThreatPilot significantly outperforms LLM-generated or the open-sourced rules with all metrics, showcasing great attack adaptability and detection abilities. Specifically, ThreatPilot achieves the highest F1 score of 0.839, with a precision of 0.815 and a recall of 0.886 under GPT-4o-mini, outperforming other rule-sets by a margin of 4–18% across all metrics. Even under less performant LLMs like LLaMa-3, ThreatPilot still shows notable improvements over both its no-feedback variant and open-source rules. The standalone LLM-generated Sigma rules can sometimes yield better detection results than the existing open-sourced rules-set from Splunk or Sigma, demonstrating that LLMs have great potential in rule generation. ThreatPilot is better than all the standalone LLMs, and through further analyzing, we find that the gap is mainly reflected in the ability to model semantic intent and contextual adaptability. ThreatPilot can improve the pertinence and robustness of Sigma rules with complete built-in functionalities and functionally-complete logic, making it effective to detect more targeted and stealthy attacks. Existing Sigma and Splunk rules remain static, detecting generic malicious scripts or files and making them less effective in dynamic malicious activities.

Case Study. We conducted a case study analyzing both the malicious activities it uniquely detected and the instances it failed to identify. For uniquely detected cases, ThreatPilot successfully uncovered threat-targeted behaviors such as privilege abuse, sensitive file access (e.g., reading/writing to `ADMIN`), and output redirection to remote paths

Table 7. Real-world anomaly detection performance across different rule sets. The logs were collected from a honeypot that captured live attacks over a three-month period.

Rules-set (Size)		Precision	Recall	F1
GPT 4o-mini	No ATI/Feedback (86)	0.588	0.774	0.668
	With ATI (235)	0.787	0.831	0.808
	With AIT/Feedback (235)	0.815	0.886	0.839
DeepSeek-V3	No ATI/Feedback (68)	0.598	0.471	0.527
	With ATI (249)	0.701	0.769	0.733
	With AIT/Feedback (249)	0.762	0.831	0.795
LLaMa-3	No ATI/Feedback (73)	0.614	0.297	0.400
	With ATI (274)	0.617	0.384	0.473
	With AIT/Feedback (274)	0.697	0.471	0.562
Splunk [41]: Application and web (121)		0.449	0.618	0.520
Sigma [38]: Emerging threat rules (366)		0.350	0.583	0.483



(a) Comparison of Sigma rules with/without ATIs.

(b) Comparison of commands with/without procedures.

Fig. 7. Comparison of Sigma rules and commands with/without additional information.

like `127.0.0.1\\ADMIN$`, which may indicate data exfiltration or reverse shell attempts. Unlike traditional static rules that typically flag generic payloads (e.g., Build.bat), ThreatPilot precisely identifies threat-critical artifacts such as `Invoke-Expression`, `DestinationIp`, and `.env`, `.git/config`, `/phpMyAdmin/index.php` from more empirical observations, due to its deep modeling of attack intent, semantic context, and integration of environmental feedback. Moreover, ThreatPilot exhibits robustness to feature-shift attacks by detecting variants that static rules-set often miss, demonstrating its intelligence and adaptability in rule generation. In contrast, the undetected cases often involve stealthy attacks with entirely novel signatures or behaviors that differ significantly from known patterns, causing both static matching and semantic reasoning methods to miss them. Finally, we simulate system attacks using logs labeled with ground-truth malicious commands. On average, the rules generated by ThreatPilot consistently achieve higher F1 scores.

4.3.2 Command Generator. We selected the useful 945 higher-rated procedures to generate corresponding commands. As shown in Table 8, comparing to those without procedures, the commands generated from procedures exhibit a higher executable rate. We attribute this to the fact that standardized procedures help mitigate LLM hallucinations, leading to more executable commands, whereas unstructured reports make it challenging for LLMs to comprehend, thereby

Table 8. Execution success of the generated commands.

Commands (Size)		Executable Percentage
GPT 4o-mini	No Procedure/Feedback (204)	53.4%
	With Procedure (945)	86.6%
	With Procedure/Feedback (945)	99.3%
DeepSeek-V3	No Procedure/Feedback (251)	55.0%
	With Procedure (879)	93.4%
	With Procedure/Feedback (879)	96.1%
LLaMa-3	No Procedure/Feedback (218)	50.5%
	With Procedure (804)	86.4%
	With Procedure/Feedback (804)	93.4%

leading to the failure of execution. Moreover, Figure 7(b) presents an example illustrating our interpretation: procedures processed by GraphRAG contain significantly more entities, resulting in more meaningful commands compared to the commands generated by unstructured reports.

4.4 RQ4: Efficiency

We use CTI reports with an average length of 408 words for the efficiency evaluation, .

Latency for ATI Extractor. We show the latency analysis and the economic costs for identifying techniques of ThreatPilot in Table 9. ThreatPilot extracts accurate intelligence from CTI reports within seconds, demonstrating its training-free and lightweight design—well-suited for processing live streams of threat intelligence data.

Model	Prompt Tokens	Output Tokens	Cost	Query Time
GPT-4o-mini	96,481	2,533	$0.020	56s
GPT-4o	91,642	2,068	$0.488	58s
DeepSeek-V3	173,846	4,187	–	61s
LLaMa-3	65,313	1,584	–	73s

Table 9. Efficiency of technique identification per CTI report.

Tasks		Ave. Rules/Commands	Cost	Time
GPT 4o-mini	Sigma Rules	18	$0.038	212s
	Commands	24	$0.045	246s
DeepSeek-V3	Sigma Rules	17	–	225s
	Commands	20	–	307s
LLaMa-3	Sigma Rules	18	–	241s
	Commands	25	–	289s

Table 10. Efficiency of end-to-end generation of Sigma rules/commands per CTI report.

End-to-end Latency Analysis for Sigma Rule Generator. Table 10 shows an end-to-end efficiency for generating Sigma rules and executable commands from a CTI report. ThreatPilot significantly outperforms manual efforts in terms of both latency and cost-efficiency, as the effort required for producing detection rules is highly dependent on the analyst's expertise and the complexity of the report [18, 35].

5 Discussion

Creativity vs. Hallucination. When using LLMs, a key trade-off is balancing creativity and hallucination. Creativity in LLMs allows ThreatPilot to generate unseen techniques. This capacity is particularly valuable in finding new ways between attack actions. However, the same mechanisms that drive creativity also increase the risk of hallucinations. We manage this trade-off in ThreatPilot by the combination of *knowledge enhancer* and *validator*. Hallucination is a

pervasive concern in LLM-assisted work [9, 29]. ThreatPilot is designed to mitigate this issue while acknowledging its inevitability. This robustness stems from the dual approach of creative generation followed by a fact-checking process, allowing it suitable for tasks for both creativity and reliability. Specifically, ThreatPilot combines a vector database which stores verified information and known techniques along with an *LLM-as-a-judge* component.

Why not Fine-tune LLMs. We opt for LLM augmentation over fine-tuning due to its practical advantages and observed limitations. A preliminary fine-tuning on a dataset of <techniques: descriptions> from `MITRE ATT&CK` yielded suboptimal results in identifying complete techniques. We attribute this to the low quality and limited coverage of the fine-tuning dataset. Furthermore, fine-tuning adjusts all model parameters and thus requires exceptionally high-quality data, which is often hard to obtain in practice. In contrast, augmentation allows LLMs to retrieve and incorporate external domain knowledge dynamically, making it a lightweight yet effective solution. It is also more resource-efficient: fine-tuning typically demands significant computational power (e.g., GPUs or paid APIs) and incurs high costs. For example, OpenAI charges $25 per million tokens for fine-tuning GPT-4o. Given the fast-evolving nature of security knowledge, maintaining up-to-date fine-tuned models becomes costly and impractical, making augmentation the more sustainable approach.

LLMs for Threat Intelligence. Recent works [46, 47] have explored the use of LLMs to enhance threat intelligence extraction. However, they often fall short in bridging the gap between extracted intelligence and its practical application. In this paper, we take a first step toward operationalizing threat intelligence by linking it to the generation of Sigma rules and executable commands, thereby demonstrating its practical value in real-world scenarios.

6 Conclusion

In this paper, we present ThreatPilot, a system for extracting and enriching attack-level threat intelligence (ATI) from a CTI report. The comprehensive ATI can guide the automatic generation of Sigma rules and commands efficiently for attack defense and reproduction, eliminating the need for analysts to manually adhere to strict syntax rules while ensuring sufficient validation. We perform a comprehensive evaluation of ThreatPilot, evaluating the accuracy of extracted techniques and the execution success of generated Sigma rules and commands. ThreatPilot outperforms state-of-the-art models like AttackKG by 1.34×. Moreover, our generated rules significantly surpass existing rule-sets in detecting real-world malicious events, and our generated commands achieve an execution success rate of 99.3%, compared to 50.3% without the intelligence guided.

References

[1] [n. d.]. CTI Reports on APT1 attacks. https://services.google.com/fh/files/misc/mandiant-apt1-report.pdf.
[2] 2018. DARPA Transparent Computing (TC) program: Engagement 3.
[3] 2020. The Top Ten MITRE ATT&CK Techniques. https://www.picussecurity.com/resource/the-top-ten-mitre-attck-techniques.
[4] 2022. text-embedding-ada-002. https://platform.openai.com/docs/guides/embeddings/what-are-embeddings.
[5] 2022. Top 10 Most Common MITRE Tactics& Techniques. https://redcanary.com/threat-detection-report/techniques/.
[6] 2024. DeepSeek V3. https://huggingface.co/deepseek-ai/DeepSeek-V3.
[7] 2024. Vulnerability Reports in Cisco Talos. https://talosintelligence.com/vulnerability_reports#disclosed.
[8] 2024. Vulnerability Research. https://talosintelligence.com/vulnerability_info.
[9] Patrice Béchard and Orlando Marquez Ayala. 2024. Reducing hallucination in structured outputs via Retrieval-Augmented Generation. *arXiv preprint arXiv:2404.08189* (2024).
[10] Sandeep N. Bhatt, Pratyusa K. Manadhata, and Loai Zomlot. 2014. The Operational Role of Security Information and Event Management Systems. *IEEE Secur. Priv.* 12, 5 (2014), 35–41. doi:10.1109/MSP.2014.103
[11] Yutong Cheng, Osama Bajaber, Saimon Amanuel Tsegai, Dawn Song, and Peng Gao. 2025. CTINexus: Automatic Cyber Threat Intelligence Knowledge Graph Construction Using Large Language Models. arXiv:2410.21060 [cs.CR] https://arxiv.org/abs/2410.21060

[12] The MITRE Corporation. 2024. The ATT&CK knowledge base is used as a foundation for the development of specific threat models and methodologies in the private sector, in government, and in the cybersecurity product and service community. https://attack.mitre.org/.

[13] Jiangyi Deng, Xinfeng Li, Yanjiao Chen, Yijie Bai, Haiqin Weng, Yan Liu, Tao Wei, and Wenyuan Xu. 2024. RACONTEUR: A Knowledgeable, Insightful, and Portable LLM-Powered Shell Command Explainer. *CoRR* abs/2409.02074 (2024). doi:10.48550/ARXIV.2409.02074 arXiv:2409.02074

[14] DShield Honeypot [n. d.]. https://isc.sans.edu/honeypot.html. DShield Honeypot.

[15] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. 2024. From Local to Global: A Graph RAG Approach to Query-Focused Summarization. *CoRR* abs/2404.16130 (2024). doi:10.48550/ARXIV.2404.16130 arXiv:2404.16130

[16] Luyu Gao, Jianmo Ni, Xuezhi Wang, Yiming Yang, Donald Metzler, and Denny Zhou. 2023. Lost in the Middle: How Language Models Use Long Contexts. *arXiv preprint arXiv:2307.03172* (2023). https://arxiv.org/abs/2307.03172

[17] Peng Gao, Fei Shao, Xiaoyuan Liu, Xusheng Xiao, Zheng Qin, Fengyuan Xu, Prateek Mittal, Sanjeev R. Kulkarni, and Dawn Song. 2021. Enabling Efficient Cyber Threat Hunting With Cyber Threat Intelligence. In *37th IEEE International Conference on Data Engineering, ICDE 2021, Chania, Greece, April 19-22, 2021*. IEEE, 193–204. doi:10.1109/ICDE51399.2021.00024

[18] Dan Harper and Richard Mabe. 2020. The Cost of Cybersecurity for IT Systems. https://www.dhs.gov/sites/default/files/publications/cyber_cost_brief_it_cast_25_aug_2020_harper_mabe.pdf.

[19] Ghaith Husari, Ehab Al-Shaer, Mohiuddin Ahmed, Bill Chu, and Xi Niu. 2017. TTPDrill: Automatic and Accurate Extraction of Threat Actions from Unstructured Text of CTI Sources. In *Proceedings of the 33rd Annual Computer Security Applications Conference, Orlando, FL, USA, December 4-8, 2017*. ACM, 103–115. doi:10.1145/3134600.3134646

[20] Valentine Legoy, Marco Caselli, Christin Seifert, and Andreas Peter. 2020. Automated Retrieval of ATT&CK Tactics and Techniques for Cyber Threat Reports. *CoRR* abs/2004.14322 (2020). arXiv:2004.14322 https://arxiv.org/abs/2004.14322

[21] Patrick S. H. Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*, Hugo Larochelle, Marc'Aurelio Ranzato, Raia Hadsell, Maria-Florina Balcan, and Hsuan-Tien Lin (Eds.). https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html

[22] Zihao Li, Zhuoran Yang, and Mengdi Wang. 2023. Reinforcement Learning with Human Feedback: Learning Dynamic Choices via Pessimism. *CoRR* abs/2305.18438 (2023). doi:10.48550/ARXIV.2305.18438 arXiv:2305.18438

[23] Zhenyuan Li, Jun Zeng, Yan Chen, and Zhenkai Liang. 2022. AttacKG: Constructing Technique Knowledge Graph from Cyber Threat Intelligence Reports. In *Computer Security - ESORICS 2022 - 27th European Symposium on Research in Computer Security, Copenhagen, Denmark, September 26-30, 2022, Proceedings, Part I (Lecture Notes in Computer Science, Vol. 13554)*, Vijayalakshmi Atluri, Roberto Di Pietro, Christian Damsgaard Jensen, and Weizhi Meng (Eds.). Springer, 589–609. doi:10.1007/978-3-031-17140-6_29

[24] Zhenyuan Li, Jun Zeng, Yan Chen, and Zhenkai Liang2. 2022. AttackKG open-sourced ground truth.

[25] Xiaojing Liao, Kan Yuan, XiaoFeng Wang, Zhou Li, Luyi Xing, and Raheem A. Beyah. 2016. Acing the IOC Game: Toward Automatic Discovery and Analysis of Open-Source Cyber Threat Intelligence. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, Vienna, Austria, October 24-28, 2016*, Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi (Eds.). ACM, 755–766. doi:10.1145/2976749.2978315

[26] Xinwei Long, Jiali Zeng, Fandong Meng, Zhiyuan Ma, Kaiyan Zhang, Bowen Zhou, and Jie Zhou. 2024. Generative Multi-Modal Knowledge Retrieval with Large Language Models. In *Thirty-Eighth AAAI Conference on Artificial Intelligence, AAAI 2024, Thirty-Sixth Conference on Innovative Applications of Artificial Intelligence, IAAI 2024, Fourteenth Symposium on Educational Advances in Artificial Intelligence, EAAI 2014, February 20-27, 2024, Vancouver, Canada*, Michael J. Wooldridge, Jennifer G. Dy, and Sriraam Natarajan (Eds.). AAAI Press, 18733–18741. doi:10.1609/AAAI.V38I17.29837

[27] Meta. 2024. Llama 3 8B. https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct.

[28] Kunal Mukherjee, Joshua Wiedemeier, Tianhao Wang, James Wei, Feng Chen, Muhyun Kim, Murat Kantarcioglu, and Kangkook Jee. 2023. Evading Provenance-Based ML Detectors with Adversarial System Actions. In *32nd USENIX Security Symposium, USENIX Security 2023, Anaheim, CA, USA, August 9-11, 2023*, Joseph A. Calandrino and Carmela Troncoso (Eds.). USENIX Association, 1199–1216. https://www.usenix.org/conference/usenixsecurity23/presentation/mukherjee

[29] Niels Mündler, Jingxuan He, Slobodan Jenko, and Martin Vechev. 2023. Self-contradictory hallucinations of large language models: Evaluation, detection and mitigation. *arXiv preprint arXiv:2305.15852* (2023).

[30] Marcin Nawrocki, John Kristoff, Raphael Hiesgen, Chris Kanich, Thomas C. Schmidt, and Matthias Wählisch. 2023. SoK: A Data-driven View on Methods to Detect Reflective Amplification DDoS Attacks Using Honeypots. In *8th IEEE European Symposium on Security and Privacy, EuroS&P 2023, Delft, Netherlands, July 3-7, 2023*. IEEE, 576–591. doi:10.1109/EUROSP57164.2023.00041

[31] Muslum Ozgur Ozmen, Ruoyu Song, Habiba Farrukh, and Z. Berkay Celik. 2023. Evasion Attacks and Defenses on Smart Home Physical Event Verification. In *30th Annual Network and Distributed System Security Symposium, NDSS 2023, San Diego, California, USA, February 27 - March 3, 2023*. The Internet Society. https://www.ndss-symposium.org/ndss-paper/evasion-attacks-and-defenses-on-smart-home-physical-event-verification/

[32] Eric Pauley and Patrick McDaniel. 2023. Understanding the ethical frameworks of internet measurement studies. In *2nd International Workshop on Ethics in Computer Security*.

[33] Ofir Press, Noah A. Smith, and Mike Lewis. 2022. Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation. *arXiv preprint arXiv:2212.07445* (2022). https://arxiv.org/abs/2212.07445

[34] Cyber Proof. 2024. Top 10 Attack Techniques Financial Organizations Should Monitor in 2024. https://go.cyberproof.com/hubfs/Infographic-Top-10-attack-techniques-financial-organizations-should-monitor-in-2024.pdf.

[35] Nidhi Rastogi, Sharmishtha Dutta, Alex Gittens, Mohammed J. Zaki, and Charu C. Aggarwal. 2022. TINKER: A framework for Open source Cyberthreat Intelligence. In *IEEE International Conference on Trust, Security and Privacy in Computing and Communications, TrustCom 2022, Wuhan, China, December 9-11, 2022*. IEEE, 1569–1574. doi:10.1109/TRUSTCOM56396.2022.00225

[36] Christian Rossow, Christian J. Dietrich, Chris Grier, Christian Kreibich, Vern Paxson, Norbert Pohlmann, Herbert Bos, and Maarten van Steen. 2012. Prudent Practices for Designing Malware Experiments: Status Quo and Outlook. In *IEEE Symposium on Security and Privacy, SP 2012, 21-23 May 2012, San Francisco, California, USA*. IEEE Computer Society, 65–79. doi:10.1109/SP.2012.14

[37] Kiavash Satvat, Rigel Gjomemo, and V. N. Venkatakrishnan. 2021. Extractor: Extracting Attack Behavior from Threat Reports. In *IEEE European Symposium on Security and Privacy, EuroS&P 2021, Vienna, Austria, September 6-10, 2021*. IEEE, 598–615. doi:10.1109/EUROSP51992.2021.00046

[38] Sigma contributors [n. d.]. https://github.com/SigmaHQ/sigma. Sigma – Generic Signature Format for SIEM Systems – GitHub, 2023.

[39] Splunk. 2025. splunk-sdk-python/splunklib at master. https://github.com/splunk/splunk-sdk-python/tree/master/splunklib.

[40] Splunk cisco company [n. d.]. https://shorturl.at/dgTsP. State of Security 2024: The Race to Harness AI.

[41] Splunk Open-sourced Rules [n. d.]. https://github.com/splunk/security_content/tree/develop/detections/network. Splunk Open-sourced Rules.

[42] Rafael Uetz, Marco Herzog, Louis Hackländer, Simon Schwarz, and Martin Henze. 2023. You Cannot Escape Me: Detecting Evasions of SIEM Rules in Enterprise Networks. *CoRR* abs/2311.10197 (2023). doi:10.48550/ARXIV.2311.10197 arXiv:2311.10197

[43] Qi Wang, Wajih Ul Hassan, Ding Li, Kangkook Jee, Xiao Yu, Kexuan Zou, Junghwan Rhee, Zhengzhang Chen, Wei Cheng, Carl A. Gunter, and Haifeng Chen. 2020. You Are What You Do: Hunting Stealthy Malware via Data Provenance Analysis. In *27th Annual Network and Distributed System Security Symposium, NDSS 2020, San Diego, California, USA, February 23-26, 2020*. The Internet Society. https://www.ndss-symposium.org/ndss-paper/you-are-what-you-do-hunting-stealthy-malware-via-data-provenance-analysis/

[44] Xindi Wang, Mahsa Salmani, Parsa Omidi, Xiangyu Ren, Mehdi Rezagholizadeh, and Armaghan Eshaghi. 2024. Beyond the Limits: A Survey of Techniques to Extend the Context Length in Large Language Models. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence, IJCAI 2024, Jeju, South Korea, August 3-9, 2024*. ijcai.org, 8299–8307. https://www.ijcai.org/proceedings/2024/917

[45] Xiaoju Ye, Zhichun Wang, and Jingyuan Wang. 2025. Infinite Retrieval: Attention Enhanced LLMs in Long-Context Processing. *CoRR* abs/2502.12962 (2025). doi:10.48550/ARXIV.2502.12962 arXiv:2502.12962

[46] Javier Yong, Haokai Ma, Yunshan Ma, Anis Yusof, Zhenkai Liang, and Ee-Chien Chang. 2025. AttackSeqBench: Benchmarking Large Language Models' Understanding of Sequential Patterns in Cyber Attacks. *CoRR* abs/2503.03170 (2025). doi:10.48550/ARXIV.2503.03170 arXiv:2503.03170

[47] Yongheng Zhang, Tingwen Du, Yunshan Ma, Xiang Wang, Yi Xie, Guozheng Yang, Yuliang Lu, and Ee-Chien Chang. 2024. AttacKG+:Boosting Attack Knowledge Graph Construction with Large Language Models. *CoRR* abs/2405.04753 (2024). doi:10.48550/ARXIV.2405.04753 arXiv:2405.04753

.1 Detailed Prompts and Procedure Generation

In this section, we provide the detailed prompts used in our framework and the details of procedure generations. Table 11, Table 12 and Table 14 present the detailed prompt for all the attack intelligence extraction ranging from the IoC extraction, the specific guidelines used in in-context learning abilities, and the guidelines used in the component of *LLM-as-a-judge*. Finally, to tailor the web application logs in real-world log evaluation, we slightly modify the guidelines to precisely customize the generation of Sigma rules and shown the detailed guidelines in Table 15.

Table 11. The detailed prompt for IoC extraction.

Guidelines :
1. The IoCs can include IP addresses, domain names, URLs, file hashes, etc.
2. Focus on terms that describe the nature of the attack, the software, or methods being used.
3. The output needs to be in JSON format.
4. If there is no IoC in the report, return an empty list.
The output format is as follows: {"ioc": ["IOC1", "IOC2", ...]}

Table 12. The detailed guidelines in *intention interpreter*.

Guidelines :
Tactics represent the "why" of an ATT&CK technique or sub-technique. It is the adversary's tactical goal: the reason for performing an action. For example, an adversary may want to achieve credential access.
Techniques represent 'how' an adversary achieves a tactical goal by performing an action. For example, an adversary may dump credentials to achieve credential access.
- You need to analyse the techniques used in the attack.
- Make sure that the techniques are one of the techniques above.
- The output needs to be in JSON format.

Table 13. The detailed guidelines in procedure generation.

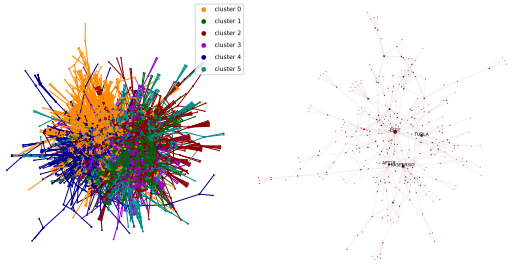
Guidelines :
Analyse the implementation details taken by the attacker to achieve their goal.
The procedure usually contains entities, actions, and relationships.
You need to focus on the entities mostly.
For example, the entities can be following:
- IP addresses or domain names such as 168.0.0.1, example.com
- File names or hashes such as ctfhost2.exe
- Programs or software names such as Mimikatz, vssadmin.exe
- And so on
The report is as follows:
{attack_report}
The technique used in the attack is{technique}.

Table 14. The detailed prompt for *LLM-as-a-judge*.

Guidelines :
- If the technique exists in the report, you need to output YES and the reason.
- If the technique does not exist in the report, you need to output NO and the reason.
- The output needs to be in JSON format. The output format is as follows:
{"if_exist": "YES/NO", "reason": "REASON"}

Table 15. The detailed guidelines for generating Sigma rules towards web applications.

Guidelines : The fields in the sigma rule should only be 'user_agent', 'extracted_source', 'url', which means the selection should be based on these fields.



(a) Clusters in GraphRAG's vector database.
(b) One example in one cluster.

Fig. 8. Demonstration of our vector database in GraphRAG.

Examples on the vector database in our GraphRAG. Clusters in our vector database of GraphRAG demonstrates the relevant entities and relationships in existing procedures. When generating the new procedures, it could first go through the knowledge graph and then generate the specific procedures. Clusters in our vector database of GraphRAG demonstrates the relevant entities and relationships in existing procedures. When generating the new procedures, it could first go through the knowledge graph and then generate the specific procedures.

.2 System Evaluation on Sigma Rule Generator

System Log Collection. To evaluate the quality of our generated Sigma rules, we simulate all the atomic attacks provided by red teams spanning 12 tactics in MITRE ATT&CK, which expected to cover all possible attacks in this tactic of attack categories. We set a max threshold of 10 for those tactic of attacks with too many atomic tests. We summarize our used system logs in Table 16 including 229,968 system logs from 61 atomic tests. The statistics are listed in Table 16, where we cover nearly all tactics of MITRE ATT&CK. The results are listed in Table 16. We only do not simulate one tactic of "resource development", because the type is a preparation for attacks including acquiring infrastructure to register domain names for malware, rather than directly executing malicious acts. We also open-sourced all the simulated datsets for public use. We use EventViewer to collect the system logs including system, sysmon, and PowerShell logs in a virtual machine with the OS of Windows 10 (64-bit). Our evaluation focuses primarily on Windows events due to their widespread use in both enterprises and consumer markets. Besides, the sophisticated attacks are mostly designed for Windows platforms. Note that our results are suitable for other systems like Linux or FreeBSD.

Metrics. We ingest all collected logs into Splunk, execute the rules as search queries on the indexed logs, and verify the accuracy of the generated alarms by comparing them against the ground truth. For each atomic test associated with a tactic, we gather the corresponding attack reports from sources such as MITRE ATT&CK and relevant web corpora. These reports are used to generate rules both with and without TTPs, without restricting the size of the generated rules. On average, each atomic test produces around 10 rules. For each tactic, we input all the generated rules—amounting to hundreds per tactic—into Splunk's execution window using a combined connector. We then compare all generated alerts with the ground truth associated with that tactic's attacks. Precision, recall, and F1 score are calculated using an

Table 16. Summary of our simulated attack logs, comprising 229,968 system logs upon 61 atomic tests. We simulate all tactics in MITRE ATT&CK, except for resource development due to its largely attack-irrelevant features.

TTP attacks	Reconnaissance T1592.001- Gather Victim Host Information	Resource Development	Initial Access T1195- Supply Chain Compromise	Execution T1053.005- Scheduled Task/Job	Persistence T1197- BITS Jobs	Privilege Escalation T1055.012- Process Injection Process Hollowing	Defense Evasion T1112- Modify Registry
Atomic Tests	1	0	1	11	4	4	10
Number of Logs	1,438	0	10,091	47,269	5,761	46,375	31,560
TTP attacks	Credential Access T1555- Credential from Password Stores	Discovery T1615- Group Policy Discovery	Lateral Movement T1021.002- Remote Services	Collection T1557.001- Adversary-in-the-Middle: LLMNR/NBT-NS Poisoning and SMB Relay	Command and Control T1071.001- Application Layer Protocol :Web Protocols	Exfiltration T1041- Exfiltration Over C2 Channel	Impact T1490- Inhibit System Recovery
Atomic Tests	6	5	4	1	3	2	10
Number of Logs	15,798	16,189	8,752	1,910	5,530	5,954	33,341

exact-match criterion. Additionally, we measure the number of attempts required to pass the grammar check as another indicator of the quality of the initial rule generation.

Analysis Results. With precise TTPs, the effectiveness of the generated rules can be largely improved, in terms of the precision, recall, and attempts to pass the rules' grammar check. The reasons could be that the detailed TTPs can give a full and organized understanding of the attacks, which can facilitate LLMs to generate better rules. Furthermore, we empirically investigate the separate gains of the module of *LLM-as-a-judge* in our end-to-end rule generation approach. We design this module to reject rules generated in previous steps, focusing on reducing false positives. Empirical cases show that the filtered rules are often overly general and attack-irrelevant. For example, a rule like *Image IN ("*CalculatorApp.exe", "*hollow_process_name.exe")* is too broad, as it only matches image names without specifically considering the process injection. As a result, the corresponding logs it captures are benign activities, such as loading *CalculatorApp.exe*.

Still, despite our efforts to use precise TTPs to reduce false negatives, critical in industrial operational environments, we are motivated for deep reasons on the remaining false negatives. When we look at several cases, we empirically conclude that the gaps stem from the inherent limitations of Sigma rules. Generating more advanced rules with conditions might address this issue, which we identify as an area for future exploration. While Sigma rules are compatible with any SIEM vendor, they primarily rely on single-pattern matching through regular expressions. They lack support for complex queries, such as SQL-style aggregations, which are necessary for statistical analysis or advanced detection. For example, when an attacker attempts to exfiltrate sensitive files by downloading multiple '.zip' files from a server, the Sigma rule identifies requests for '.zip' files. However, it is limited to basic string matching and cannot account for contextual factors, such as excessive downloads or filtering benign activities based on user agent strings.